

Learn Google Earth Engine

全套教学课程

面向遥感、GIS、生态与林业科研人员的 JavaScript Code Editor 实战手册

2026.07

使用说明

建议按“阅读知识点 → 复制代码 → 点击 Run → 对照结果 → 主动修改参数”的顺序学习。

完成课程后，你应当能够：

1. 独立完成 GEE Cloud 项目注册、非商业资格配置与 Code Editor 准入。
2. 看懂 Code Editor 左、中、右与地图区域，能管理脚本、资产和导出任务。
3. 正确区分 Geometry、Feature、FeatureCollection、Image 与 ImageCollection。
4. 完成 Sentinel-2 和 Landsat 数据的筛选、去云、缩放、裁剪与合成。
5. 计算 NDVI、EVI、NDBI、NDWI、SAVI，绘制时序图并导出成果。
6. 完整复现 2018-2026 森林 NDVI 时序趋势项目，并理解其科研假设与局限。
7. 独立上传 Table/Image Asset，并把私有资产安全地用于整体 ROI 或逐区统计。
8. 设计连续、分类、RGB、矢量、图例和图表等可验证的可视化。
9. 正确选择脚本链接、GitHub、Asset 权限或 Earth Engine App 进行协作与公开发布。

课程目录

目录

使用说明.....	2
课程目录.....	3
1. GEE 基础介绍与平台准入.....	9
1.1 GEE 是什么、为什么值得学	9
1.2 云端计算原理.....	11
1.3 账号注册、授权与进入 Code Editor	13
1.4 注册失败、权限与账号问题排查.....	16
1.5 学习路线与核心数据类型概览.....	17
2 GEE Code Editor 教学	19
2.1 print、点线面矩形与 Map.addLayer.....	19
7.3 逐行阅读重点	错误!未定义书签。
7.4 结果说明	20
7.5 知识点总结	错误!未定义书签。
7.6 常见报错解决	错误!未定义书签。
第 8 章 五类核心数据结构逐套案例	20
8.1 知识点讲解.....	20
8.2 完整可运行代码	错误!未定义书签。
8.3 逐行阅读重点	21
8.4 结果说明	22
8.5 知识点总结	错误!未定义书签。
8.6 常见报错解决	22
第 9 章 筛选、去云、镶嵌与裁剪	22
9.1 知识点讲解.....	22
9.2 完整可运行代码	错误!未定义书签。
9.3 结果说明	24
9.4 知识点总结	错误!未定义书签。
9.5 常见报错解决	24

第 10 章 NDVI、EVI、NDBI、NDWI 与 SAVI	24
10.1 指数原理	24
10.2 可运行代码.....	24
10.3 结果说明	25
10.4 知识点总结.....	错误!未定义书签。
10.5 常见报错解决.....	25
第 11 章 均值、最大值、最小值与月度/年度合成.....	25
11.1 知识点讲解.....	25
11.2 完整可运行代码.....	25
11.3 逐行阅读重点.....	28
11.4 结果说明	28
11.5 知识点总结.....	错误!未定义书签。
11.6 常见报错解决.....	28
第 12 章 图层配色、可视化与 Inspector 判读.....	28
12.1 知识点讲解.....	28
12.2 可运行代码.....	28
12.3 结果说明	29
12.4 知识点总结.....	29
12.5 常见报错解决.....	29
第 13 章 ui.Chart 图表绘制	29
13.1 知识点讲解.....	29
13.2 可运行代码.....	29
13.3 结果说明	30
13.4 知识点总结.....	错误!未定义书签。
13.5 常见报错解决.....	30
第 14 章 导出影像、矢量和表格.....	30
14.1 知识点讲解.....	30
14.2 可运行代码.....	30
14.3 结果说明	31
14.4 知识点总结.....	错误!未定义书签。
14.5 常见报错解决.....	32
第 15 章 客户端/服务器、map 与性能调试.....	32

15.1 知识点讲解.....	32
15.2 错误与正确示例.....	32
15.3 结果说明	32
15.4 知识点总结.....	错误!未定义书签。
15.5 常见报错解决.....	33
第 16 章 模块 2 综合练习与验收.....	33
16.1 任务	33
16.2 验收标准	33
16.3 知识点总结.....	错误!未定义书签。
16.4 常见报错解决.....	33
模块 2 · 进阶专题 从个人数据到公开应用与 AI 辅助编程.....	错误!未定义书签。
专题 A 上传、检查与管理个人 Asset.....	错误!未定义书签。
A.1 Asset 是什么.....	错误!未定义书签。
A.2 上传 Shapefile 或 CSV 为 Table Asset.....	错误!未定义书签。
A.3 上传 GeoTIFF 为 Image Asset.....	错误!未定义书签。
A.4 Asset 权限与管理.....	错误!未定义书签。
A.5 结果说明.....	错误!未定义书签。
A.6 知识点总结	错误!未定义书签。
A.7 常见报错解决	错误!未定义书签。
专题 B 把个人 Asset 用作 ROI 与分区统计	错误!未定义书签。
B.1 先区分 FeatureCollection 与 Geometry	错误!未定义书签。
B.2 属性筛选与防空检查.....	错误!未定义书签。
B.3 整体 ROI、逐区统计与几何复杂度	错误!未定义书签。
B.4 完整可运行示例	错误!未定义书签。
B.5 结果说明	错误!未定义书签。
B.6 知识点总结	错误!未定义书签。
B.7 常见报错解决	错误!未定义书签。
专题 C 建立可解释、可比较的可视化	错误!未定义书签。
C.1 可视化与分析值分离.....	错误!未定义书签。
C.2 五类常用方法	错误!未定义书签。
C.3 Inspector、直方图与统计三重检查	错误!未定义书签。
C.4 图例原则	错误!未定义书签。

C.5 结果说明	错误!未定义书签。
C.6 知识点总结	错误!未定义书签。
C.7 常见报错解决	错误!未定义书签。
专题 D 共享脚本、GitHub、Assets 与公开 Earth Engine App	错误!未定义书签。
D.1 四种共享方式解决不同问题	错误!未定义书签。
D.2 共享脚本给协作者	错误!未定义书签。
D.3 用 GitHub 做可引用的公开项目	错误!未定义书签。
D.4 发布 Earth Engine App	错误!未定义书签。
D.5 完整 App 模板	错误!未定义书签。
D.6 结果说明	错误!未定义书签。
D.7 知识点总结	错误!未定义书签。
D.8 常见报错解决	错误!未定义书签。
专题 E 六遍读懂一段 GEE 代码	错误!未定义书签。
E.1 阅读目标	错误!未定义书签。
E.2 六遍阅读法	错误!未定义书签。
E.3 带类型标注的阅读练习	错误!未定义书签。
E.4 读错代码的三个信号	错误!未定义书签。
E.5 结果说明	错误!未定义书签。
E.6 知识点总结	错误!未定义书签。
E.7 常见报错解决	错误!未定义书签。
专题 F GPT × GEE 的 Vibe Coding: 从提示词到科学验收	错误!未定义书签。
F.1 正确定义 Vibe Coding	错误!未定义书签。
F.2 提示词的八项任务契约	错误!未定义书签。
F.3 一个高质量生成提示词	错误!未定义书签。
F.4 调试提示词: 给证据, 不给情绪	错误!未定义书签。
F.5 三层真实性核验	错误!未定义书签。
F.6 完整最小验收脚本	错误!未定义书签。
F.7 结果正确性检查表	错误!未定义书签。
F.8 数据安全性与作者责任	错误!未定义书签。
F.9 结果说明	错误!未定义书签。
F.10 知识点总结	错误!未定义书签。
F.11 常见报错解决	错误!未定义书签。

模块 3 完整科研实战：2018—2026 森林植被动态监测	错误!未定义书签。
第 17 章 项目设计、研究问题与数据选择	错误!未定义书签。
17.1 项目主题	错误!未定义书签。
17.2 研究问题	错误!未定义书签。
17.3 数据选择	错误!未定义书签。
17.4 2026 年可比性设计	错误!未定义书签。
17.5 知识点总结	错误!未定义书签。
17.6 常见报错解决	错误!未定义书签。
第 18 章 研究区、分区与森林样本框	错误!未定义书签。
18.1 知识点讲解	错误!未定义书签。
18.2 可运行代码	错误!未定义书签。
18.3 结果说明	错误!未定义书签。
18.4 知识点总结	错误!未定义书签。
18.5 常见报错解决	错误!未定义书签。
第 19 章 Sentinel-2/Landsat 批量获取与预处理	错误!未定义书签。
19.1 Sentinel-2 路线	错误!未定义书签。
19.2 Landsat 路线	错误!未定义书签。
19.3 可运行核心代码	错误!未定义书签。
19.4 结果说明	错误!未定义书签。
19.5 知识点总结	错误!未定义书签。
19.6 常见报错解决	错误!未定义书签。
第 20 章 年度最大 NDVI 合成	错误!未定义书签。
20.1 原理	错误!未定义书签。
20.2 可运行核心代码	错误!未定义书签。
20.3 结果说明	错误!未定义书签。
20.4 知识点总结	错误!未定义书签。
20.5 常见报错解决	错误!未定义书签。
第 21 章 线性趋势、Sen' s slope 与 Kendall tau	错误!未定义书签。
21.1 指标解释	错误!未定义书签。
21.2 可运行核心代码	错误!未定义书签。
21.3 结果说明	错误!未定义书签。
21.4 知识点总结	错误!未定义书签。

21.5 常见报错解决.....	错误!未定义书签。
第 22 章 变化提取与各区域植被占比	错误!未定义书签。
22.1 变化分类	错误!未定义书签。
22.2 面积与占比原理.....	错误!未定义书签。
22.3 结果说明	错误!未定义书签。
22.4 知识点总结.....	错误!未定义书签。
22.5 常见报错解决.....	错误!未定义书签。
第 23 章 可视化、导出与论文方法表述.....	错误!未定义书签。
23.1 成果清单	错误!未定义书签。
23.2 论文方法段落模板	错误!未定义书签。
23.3 科研质量控制清单	错误!未定义书签。
23.4 知识点总结.....	错误!未定义书签。
23.5 常见报错解决.....	错误!未定义书签。
第 24 章 完整项目代码与复现实验清单.....	错误!未定义书签。
24.1 完整可直接运行脚本	错误!未定义书签。
24.2 首次运行步骤.....	错误!未定义书签。
24.3 参数替换指南.....	错误!未定义书签。
24.4 最终验收	错误!未定义书签。
附录 A 课程代码文件索引	错误!未定义书签。
附录 B 高频错误速查	错误!未定义书签。
附录 C 官方资料与数据引用.....	错误!未定义书签。
附录 D GitHub 教程参考与真实性核验说明	错误!未定义书签。
D.1 参考过的公开教程项目	错误!未定义书签。
D.2 本课程的核验层级	错误!未定义书签。
D.3 “代码已验证”的准确含义	错误!未定义书签。

1. GEE 基础介绍与平台准入

1.1 GEE 是什么、为什么值得学

1.1.1 GEE 介绍

Google Earth Engine (GEE) 是面向地球科学与地理空间分析的云平台，其对应的网址是：<https://code.earthengine.google.com>。它把大规模公开遥感数据目录、并行计算能力、地理空间算法和在线开发环境放在一起。用户通常不是先下载几百景影像再在本地逐景处理，而是向服务器描述“需要哪些数据、执行什么计算、输出什么结果”。

GEE 的主要优势包括：

1. 数据集中：Landsat、Sentinel、MODIS、气候、地形、土地覆盖等数据可直接调用。
2. 计算靠近数据：筛选、合成和统计在云端完成，降低本地存储和预处理负担。
3. 可重复：代码保存了数据集 ID、时间、空间范围、算法与参数。
4. 易扩展：从一个样地改到一个省，通常只需替换研究区并评估计算规模。
5. 易分享：脚本、App、导出表和影像可以形成完整复现链。

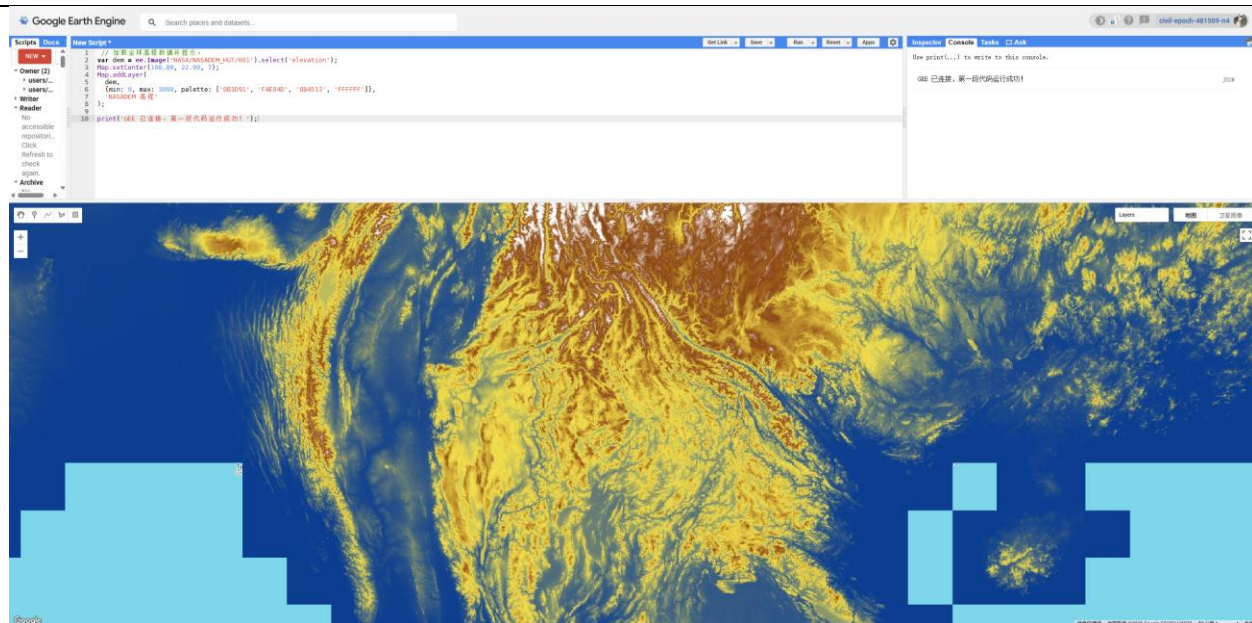
典型应用场景包括森林变化、植被物候、火烧迹地、干旱、农作物、水体、城市扩张、生态恢复、气候暴露、地表温度和灾害监测。GEE 擅长大范围、多时相、规则明确的栅格运算；它不是桌面 GIS 的完全替代品。复杂制图排版、精细人工编辑、离线保密环境和超定制本地模型仍可能需要 QGIS、ArcGIS、R 或 Python。



1.1.2 应用示例

可运行代码如下：

```
// 加载全球高程数据并显示。
var dem = ee.Image('NASA/NASADEM_HGT/001').select('elevation');
Map.setCenter(100.80, 22.00, 7);
Map.addLayer(
  dem,
  {min: 0, max: 3000, palette: ['0B3D91', 'F4E04D', '8B4513', 'FFFFFF']},
  'NASADEM 高程'
);
print('GEE 已连接，第一段代码运行成功！');
```



结果说明：运行成功后，Console 中会出现成功提示，地图显示彩色高程。值得注意的是，GEE 中的数据并没有下载到本机，浏览器只收到当前视窗需要的地图瓦片。

1.1.3 为什么选择 GEE?

GEE 平台的建立旨在加速地球科学数据的处理和分析，它提供了大量的卫星图像数据以及丰富的地理空间分析工具，借助 Google 强大的算力，GEE 可以对全球地理数据进行大规模并行处理。GEE 让用户从卫星图像中提取有用信息变得更加容易和高效，在其提供的在线开发环境中，用户可以编写 JavaScript 代码来定义、调用和处理地理数据，让研究者能够集中精力于科研问题本身，而不是数据处

理和存储问题。总之，GEE 不仅仅是一个数据仓库，它还是一个交互式地理信息分析工具，集成了先进的图像处理和机器学习算法，使得复杂的数据分析任务在几分钟内即可完成。

1.1.4 常见报错解决

- **Image asset ... not found**: 数据集 ID 拼写错误或数据已迁移；从 Data Catalog 复制当前 ID。
- 地图全黑：显示范围 **min/max** 不合适；先查看数据目录的数值范围或用图层设置做拉伸。
- Console 没有内容：确认点击了 Run，且代码左侧没有红色语法标记。

1.2 云端计算原理

1.2.1 云端计算原理

GEE Code Editor 是浏览器端开发环境。代码中的 **ee.Image**、**ee.ImageCollection** 等对象是服务器对象：它们保存的是计算描述，而不是已经传到浏览器的全部像元。GEE 采用惰性求值；只有显示图层、打印结果、生成图表或提交导出等请求，计算图才会被实际求值。

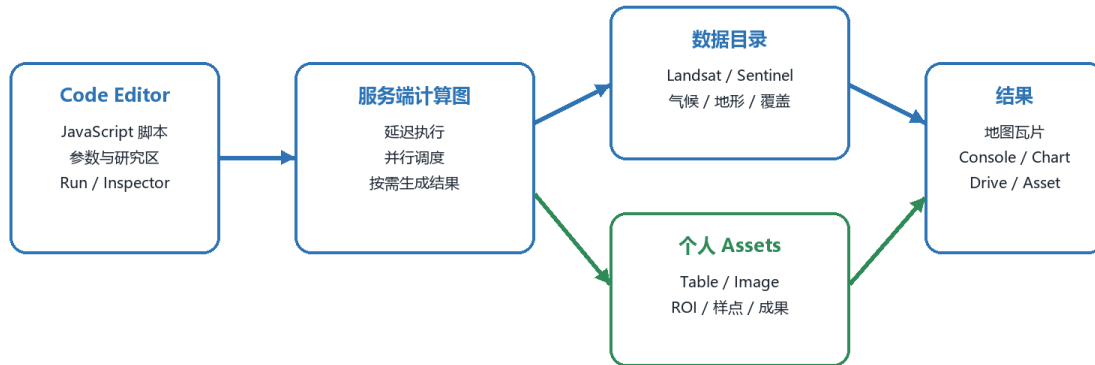
一次典型流程可以理解为：

1. 浏览器创建“筛选 2025 年 Sentinel-2”的计算描述。
2. 追加“去云、算 NDVI、取中位数、裁剪研究区”的操作。
3. 点击 Run 后，描述被发送给 Earth Engine 服务。
4. 服务器从数据目录读取所需切片，分布式执行运算。
5. 地图只返回当前视窗瓦片；统计只返回小字典；导出任务把完整成果写入 Drive、Cloud Storage 或资产。

“无需下载”并不是“数据永远不能导出”，而是分析前不必把原始影像全部搬到本地。最终论文图、GeoTIFF、CSV、SHP 仍可按需要导出。

GEE 云端计算：代码靠近数据

浏览器只提交计算描述；真正的筛选、合成与统计在 Earth Engine 服务端执行

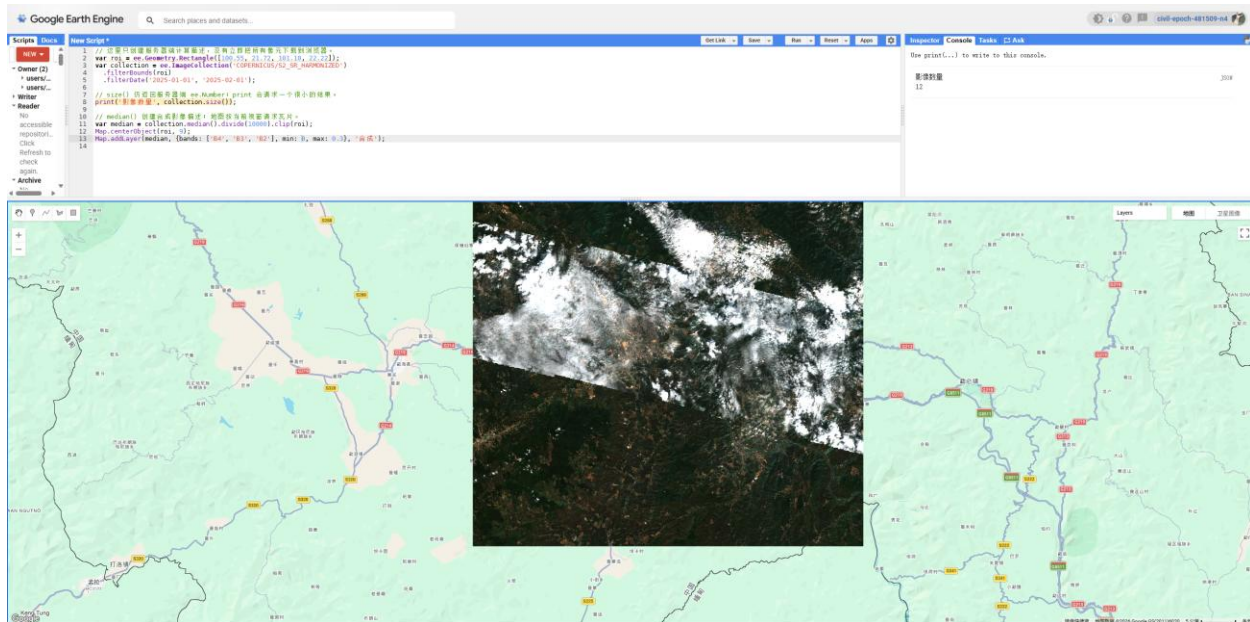


关键认识 Map.addLayer 显示的是按视窗请求的瓦片；Export 才是持久化结果。

图 1 浏览器提交计算描述，服务端按需读取公开目录或个人 Assets，再返回瓦片、统计或导出任务。

示例代码如下：

```
// 这里只创建服务器端计算描述，没有立即把所有像元下载到浏览器。  
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);  
var collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')  
  .filterBounds(roi)  
  .filterDate('2025-01-01', '2025-02-01');  
  
// size() 仍返回服务器端 ee.Number；print 会请求一个很小的结果。  
print('影像数量', collection.size());  
  
// median() 创建合成影像描述；地图按当前视窗请求瓦片。  
var median = collection.median().divide(10000).clip(roi);  
Map.centerObject(roi, 9);  
Map.addLayer(median, {bands: ['B4', 'B3', 'B2'], min: 0, max: 0.3}, '合成');
```



结果说明：Console 返回影像数量，地图显示研究区合成。浏览器没有接收完整 Sentinel-2 文件，只获得数量和可视化瓦片。先筛选再计算、尽量在服务器端链式处理、只把小结果传回客户端，是 GEE 高效工作的基本原则。

1.2.2 常见报错解决

- User memory limit exceeded: 缩小研究区、先选波段、提高统计尺度、使用 tileScale，不要盲目反复运行。
- Computation timed out: 交互请求太大；将最终计算改为 Export 批任务。
- getInfo() 卡住：它会把服务器结果同步拉到客户端；大对象不要用 getInfo()。

1.3 账号注册、授权与进入 Code Editor

1.3.1 申请项目

Earth Engine 在 Google Cloud 上运行，需要使用 Cloud 项目进行访问和 API 管理。所有 Earth Engine 调用（无论是来自代码编辑器、客户端库、应用还是 REST API）都会通过云项目进行路由，从而在 Cloud 控制台中实现访问权限控制、资源管理和使用情况监控。

获取 Earth Engine 访问权限

如需使用 Earth Engine，您需要有权访问满足以下条件的 Cloud 项目：

- 已启用 Earth Engine API，
- 已注册用于[商业](#)或[非商业](#)用途，并且
- 授予您（或用户）[正确的角色和权限](#)。

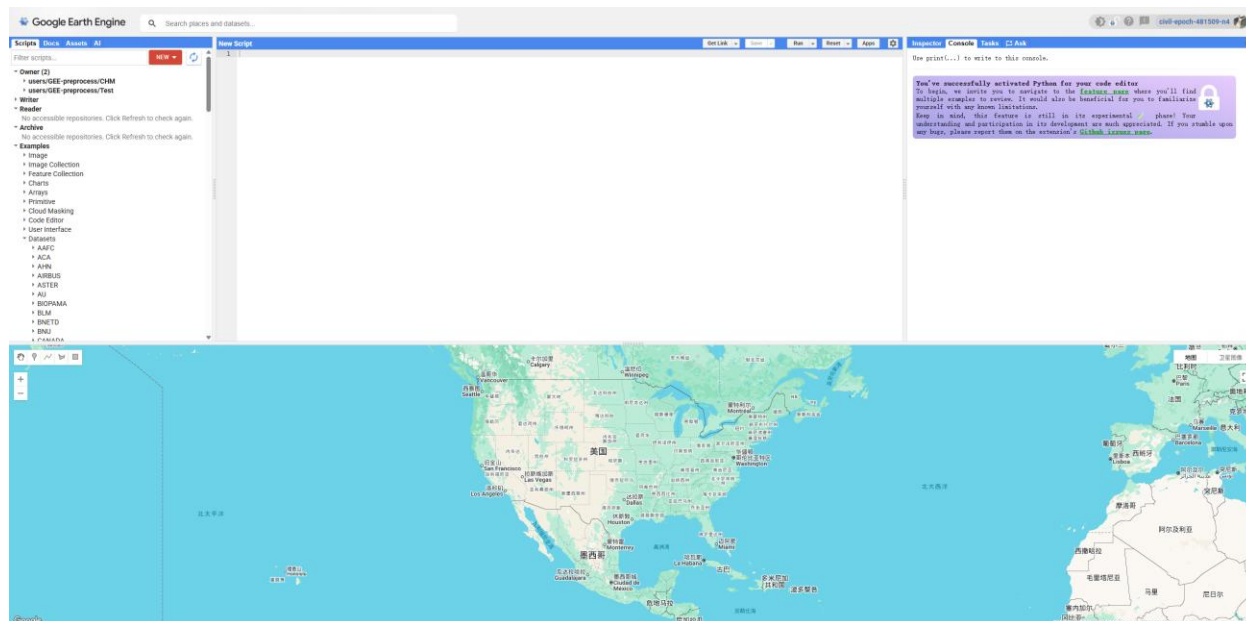
创建项目的教程见官方入口：[Earth Engine Access](#)

Earth Engine 非商业用途层级说明：[Noncommercial Tiers](#)

1.3.2 Code Editor 界面布局介绍

首次访问 Code Editor 时，您可能会看到一个导航，其中重点介绍了 Code Editor 的不同功能。Code Editor 界面布局可以划分为左侧、中间、右侧和地图四个板块。其中，左侧通常包含 Scripts、Docs、Assets 和 AI：

- **Scripts:** 基于 Git 的脚本管理器。Owner 保存自己的仓库；Writer/Reader 是他人授权的仓库；Examples 是官方示例。脚本和仓库保留版本历史，可比较或恢复。
- **Docs:** 完整 JavaScript API 文档。输入 `normalizedDifference`、`reduceRegion` 等名称可查看参数、返回类型和示例。
- **Assets:** 上传和管理栅格、表格等个人资产。上传任务完成后复制完整 Asset ID；脚本共享不等于资产自动共享。
- **AI:** AI 是 Google Earth Engine 官方推出的一个问答系统，你可以向它提问，帮助你编写、调试或理解代码，不过功能比较鸡肋。



Earth Engine 代码编辑器（网址为 code.earthengine.google.com）

中间是 JavaScript 编辑器，提供语法高亮、括号补全、API 自动完成、错误下划线和 Imports 区。GEE Code Editor 目前主要支持 ECMAScript 5 风格；课程统一使用 `var` 和普通 `function`，避免 ES6 语法兼容问题。

地图与右侧面板包括以下内容：

- **Map:** 显示影像、矢量和 UI 控件。
- **Layer Manager:** 开关图层、调整透明度、改变可视化拉伸和调色板。
- **Inspector:** 点击地图查看坐标、缩放级别、像元值和矢量属性。
- **Console:** 显示 `print()`、字典、列表、图表和报错。
- **Tasks:** 显示尚未提交、运行中和已完成的导入/导出任务。

- **Profiler:** 通过 Run 下拉菜单启用，查看算法 CPU 和内存消耗。
- **Ask:** 当前 Code Editor 中的 AI 助手入口；生成代码后仍需人工核对数据集、波段和科研假设。

此外，要介绍一下顶部和地图工具栏，这里用表格进行展示：

控件	功能	新手注意事项
Run	执行当前脚本	Run 下拉菜单可启用 Profiler
Save	保存当前脚本版本	首次保存需选择仓库和文件名
Reset	清空输出地图与 Console，恢复运行前状态	不等于删除脚本
Get Link	生成当前环境快照链接	私有资产不会因链接而自动共享
Get Link 下拉	复制脚本路径、管理历史链接	“快照链接”和“最新保存版本链接”含义不同
Search places and datasets	搜地点、公开栅格和表格数据	从结果页点击 Import 可插入脚本
Project selector	选择执行请求的 Cloud 项目	权限和配额以所选项目为准
Help/Feedback	指南、论坛、导览、快捷键和反馈	报错时先复制完整错误文本
地图点/线/面/矩形工具	交互绘制几何并创建 Imports	点坐标仍是经度在前、纬度在后
地图底图切换	更换地图背景	仅改变背景，不改变分析数据

为了你更好地熟悉界面，这里进行一个界面练习，请在脚本编辑界面键入如下代码：

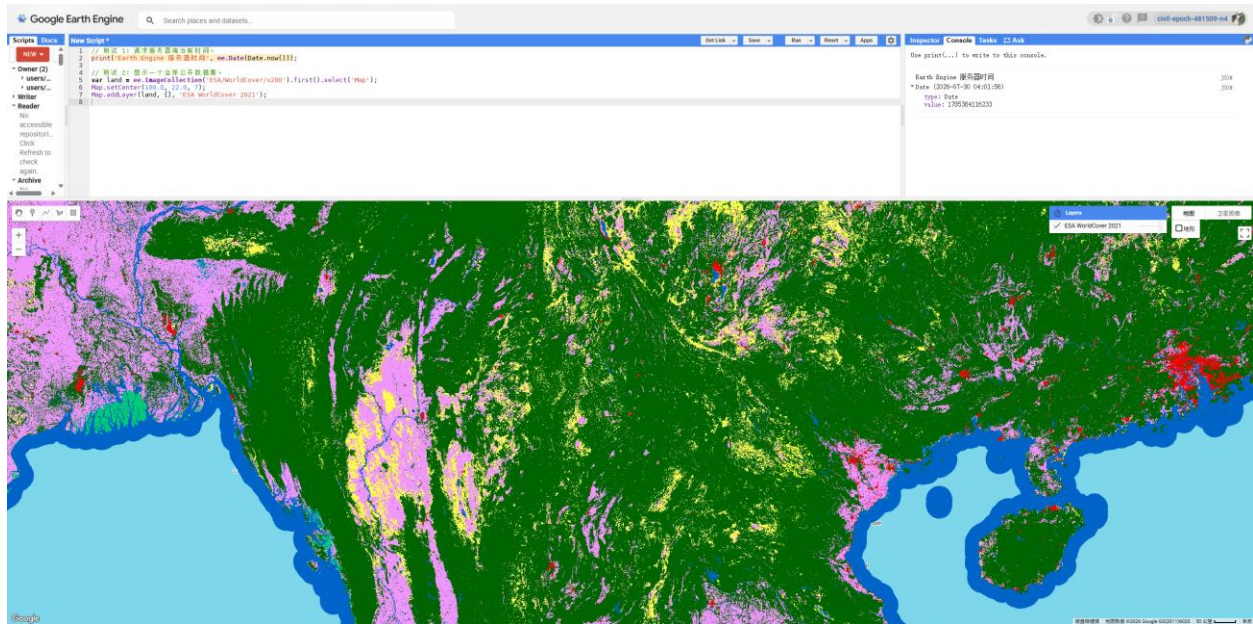
```
print('在 Console 找到这句话');
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
Map.centerObject(roi, 9);
Map.addLayer(roi, {color: 'yellow'}, '在 Layer Manager 找到这个图层');
```

运行后依次完成如下操作：在 Console 展开输出；在 Layer Manager 开关图层；启用 Inspector 点击矩形边界；打开 Docs 搜索 Map.addLayer；在 Scripts 新建并保存脚本。

此外，我们再进行一个准入测试，使用如下命令创建一个新脚本：

```
// 测试 1：请求服务器端当前时间。
print('Earth Engine 服务器时间', ee.Date(Date.now()));

// 测试 2：显示一个全球公开数据集。
var land = ee.ImageCollection('ESA/WorldCover/v200').first().select('Map');
Map.setCenter(100.8, 22.0, 7);
Map.addLayer(land, {}, 'ESA WorldCover 2021');
```



结果说明：测试代码不需要个人资产。Console 能展开 `ee.Date`，地图能显示 WorldCover，即说明账号、Cloud 项目、API 和 Code Editor 基本链路正常。

1.3.4 常见报错解决

- **Please select a Cloud Project:** 在 Code Editor 项目选择器中选择已注册项目。
- **Project is not registered for Earth Engine:** 回到 Earth Engine 配置页完成注册和用途选择。
- **403 PERMISSION_DENIED:** 确认登录账号对该 Cloud 项目有使用权限；组织项目需管理员授权。
- 非商业访问被暂停：检查资格问卷或重新验证状态，并查看项目配额是否耗尽。
- 多账号跳转错误：退出无关 Google 账号或使用独立浏览器用户配置文件重新登录。

1.4 注册失败、权限与账号问题排查

1.4.1 排查顺序

遇到错误时先保存错误全文，再按以下顺序定位：

1. 身份：浏览器当前登录的是哪个 Google 账号？
2. 项目：Code Editor 当前选择了哪个 Cloud 项目？
3. 注册：该项目是否已注册为 Earth Engine 项目？
4. API：Earth Engine API 是否启用？
5. IAM：账号是否有使用项目和 Earth Engine 资源的权限？
6. 配额：非商业月度 EECU 或并发任务限制是否达到？
7. 资产：脚本引用的个人资产是否存在且已共享？

1.4.2 典型问题与解决方案

现象	可能原因	处理方式
Code Editor 打不开或循环登录	多账号/OAuth 状态冲突	使用 Chrome 独立用户配置文件，仅登录目标账号
公开数据也报 403	项目未注册、API 未启用或 IAM 不足	检查项目配置、API 与项目权限
只有个人资产报错	Asset ID 错误或未共享	在 Assets 复制完整 ID，配置读取权限
Export 一直未开始	任务只创建未提交	在 Tasks 点击 Run；关闭页面前提交
任务运行慢	队列、规模或配额限制	查看任务详情；减少规模或等待，不要重复提交同一任务
旧脚本突然无权限	非商业验证到期或项目切换	重新验证并确认 Code Editor 当前项目

1.4.3 特别提示

阅读 Earth Engine 使用入门，使用 Earth Engine JavaScript API 进行编程，最重要的方法是会查阅官方文档（<https://developers.google.com/earth-engine/guides/>），Earth Engine 文档面向熟悉地理空间数据分析的用户。这些文档主要按数据类型进行构建。例如，左侧导航栏包含指向与重要数据类型（例如 Image、ImageCollection、Feature、FeatureCollection、Geometry、Reducer、Chart、Join 和 Array）相关的部分或页面的链接。本指南仅介绍了这些类型的相关信息，仅供您参考。还有一些部分介绍了机器学习、专用算法或特定于传感器的算法（例如 Landsat 算法）、面向公众的应用、数据（资产）管理，以及 Earth Engine 内部运作的重要详细信息。

1.5 学习路线与核心数据类型概览

1.5.1 推荐学习路线

1. 第一阶段：界面、print、Geometry、Map.addLayer。
2. 第二阶段：Feature/FeatureCollection、Image/ImageCollection。
3. 第三阶段：filter、map、mask、指数、合成、reduce。
4. 第四阶段：图表、导出、任务和性能优化。
5. 第五阶段：科研设计、可比性、趋势、验证、不确定性和复现。

本课程按五类教学：Geometry、Feature、FeatureCollection、Image、ImageCollection。FeatureCollection 是矢量集合，ImageCollection 是栅格时序集合；二者的 filter() 与 map() 形式相似，但元素类型不同。

1.5.2 一次看懂五类对象

```
// Geometry：空间形状。
```

```

var geometry = ee.Geometry.Point([100.80, 22.00]);

// Feature: 空间形状 + 属性。
var feature = ee.Feature(geometry, {name: '样地 1', type: 'forest'});

// FeatureCollection: 多个 Feature。
var featureCollection = ee.FeatureCollection([feature]);

// Image: 一张栅格影像。
var image = ee.Image('NASA/NASADEM_HGT/001').select('elevation');

// ImageCollection: 多张影像组成的集合。
var imageCollection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterDate('2025-01-01', '2025-02-01');

print('Geometry', geometry);
print('Feature', feature);
print('FeatureCollection', featureCollection);
print('Image', image);
print('ImageCollection', imageCollection);

```

在 Console 展开对象，重点观察 Geometry 的坐标、Feature 的 properties、Image 的 bands、ImageCollection 的 features/影像元数据。“单个还是集合”“矢量还是栅格”是识别对象的两个问题。能准确回答，绝大多数类型错误都能避免。

1.5.3 常见报错解决

- `... is not a function`: 常见于把 Image 当 ImageCollection 使用；先 `print()` 检查类型。
- `.first()` 后无法直接显示：用 `ee.Image(collection.first())` 明确转换，尤其在复杂表达式中。
- Feature 没有波段：Feature 是矢量对象，不能使用 `select('B4')` 读取影像波段。

2 GEE Code Editor 教学

2.1 print、点线面矩形与 Map.addLayer

2.1.1 知识点讲解

print() 用于检查文字、数字、Earth Engine 对象和图表。几何坐标使用 WGS84 经度/纬度时顺序为 [longitude, latitude]。Map.addLayer(object, visParams, name, shown, opacity) 将对象加入地图；矢量可直接添加，也可先 style() 获得更细致样式。

2.1.2 完整可运行代码

```
// =====  
// GEE 从零入门课程：基础语法、print()、几何与矢量可视化  
// 运行环境：https://code.earthengine.google.com/  
// =====  
  
// 1. print(): 把文字、数字和 Earth Engine 对象输出到 Console。  
print('你好，Google Earth Engine! ');  
print('1 + 2 =', 1 + 2);  
  
// 2. 创建一个点。坐标顺序固定为 [经度, 纬度]。  
var point = ee.Geometry.Point([100.80, 22.00]);  
  
// 3. 创建一条线。数组中的每一项都是一个 [经度, 纬度] 坐标。  
var line = ee.Geometry.LineString([  
  [100.62, 21.88],  
  [100.80, 22.00],  
  [101.02, 22.14]  
]);  
  
// 4. 创建一个面。首尾坐标相同，用于闭合多边形。  
var polygon = ee.Geometry.Polygon([  
  [100.58, 21.76],  
  [101.05, 21.76],  
  [101.05, 22.18],  
  [100.58, 22.18],  
  [100.58, 21.76]  
]);  
  
// 5. 创建一个矩形，参数依次为西、南、东、北。  
var rectangle = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);  
  
// 6. 把几何对象显示到地图。color 可写颜色名称或十六进制颜色。  
Map.addLayer(point, {color: 'red'}, '点');  
Map.addLayer(line, {color: 'yellow'}, '线');  
Map.addLayer(polygon, {color: '00FFFF'}, '面');  
Map.addLayer(rectangle, {color: 'FFFFFF'}, '矩形');  
  
// 7. 让地图自动缩放到矩形范围；数字 10 是可选缩放级别。  
Map.centerObject(rectangle, 10);  
  
// 8. 计算几何属性。maxError 用 1 米，结果转换为更直观的单位。  
print('点坐标', point.coordinates());  
print('线长度（千米）', line.length({maxError: 1}).divide(1000));  
print('多边形面积（平方千米）',  
  polygon.area({maxError: 1}).divide(1e6));  
  
// 9. 把面包装为 Feature，并添加属性。
```

```

var forestPlot = ee.Feature(polygon, {
  plot_id: 'P01',
  land_type: '教学样地',
  survey_year: 2026
});

// 10. Feature 可用 style() 设置填充色、边线颜色和线宽。
Map.addLayer(forestPlot, {
  color: '006400',
  fillColor: '66AA6666',
  width: 2
}, '带属性的样地');
print('样地 Feature', forestPlot);

```

分析代码中函数的功能：

1. `ee.Geometry.Point` 接收一对坐标。
2. `LineString` 接收有顺序的坐标列表。
3. `Polygon` 外层数组表示环；首尾相同便于直观看出闭合。
4. `Rectangle` 按西、南、东、北定义。
5. `Feature(geometry, properties)` 把空间和调查属性组合起来。
6. 面积默认平方米，除以 `1e6` 转为平方千米。

结果说明：地图出现点、线、面、矩形和样地样式层；Console 给出点坐标、线长度、多边形面积与 Feature 属性。Geometry 负责“在哪里”，Feature 负责“在哪里 + 是什么”。Map.addLayer 只控制显示，不会改变原始对象。

2.2 五类核心数据结构

2.2.1 知识点讲解

Geometry 是几何；Feature 是单个矢量要素；FeatureCollection 是矢量集合；Image 是多波段栅格；ImageCollection 是影像序列。集合的常用思路是先 filter，再 map，最后用 first/median/max/reduce 得到具体结果。

```

// =====
// GEE 从零入门课程：Geometry、Feature、FeatureCollection、
// Image 与 ImageCollection 完整示例
// =====

// ----- 1. Geometry：只有空间形状，没有业务属性 -----
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
Map.centerObject(roi, 9);
Map.addLayer(roi, {color: 'FFFFFF'}, '研究区');
print('Geometry 类型', roi);

// ----- 2. Feature：Geometry + 属性字典 -----
var plotA = ee.Feature(
  ee.Geometry.Point([100.72, 21.92]),
  {plot_id: 'A', forest_type: '常绿阔叶林', canopy: 0.82}
);
var plotB = ee.Feature(

```

```

    ee.Geometry.Point([100.91, 22.05]),
    {plot_id: 'B', forest_type: '人工林', canopy: 0.64}
);
print('Feature A', plotA);
print('A 的林型属性', plotA.get('forest_type'));

// ----- 3. FeatureCollection: 多个 Feature 的集合 -----
var plots = ee.FeatureCollection([plotA, plotB]);
Map.addLayer(plots.style({color: 'FF00FF', pointSize: 6}), {}, '样地点');
print('FeatureCollection', plots);
print('样地数量', plots.size());
print('平均郁闭度', plots.aggregate_mean('canopy'));

// 属性筛选: 保留 canopy 大于 0.7 的样地。
var densePlots = plots.filter(ee.Filter.gt('canopy', 0.7));
print('高郁闭度样地', densePlots);

// ----- 4. Image: 具有一个或多个波段的栅格影像 -----
// 从 Sentinel-2 地表反射率集合中筛选并取得第一景影像。
var s2One = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate('2025-01-01', '2025-04-01')
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
    .sort('CLOUDY_PIXEL_PERCENTAGE')
    .first();

// 打印波段和元数据; 影像波段值按 10000 缩放。
print('单张 Sentinel-2 Image', s2One);
print('单张影像波段', s2One.bandNames());
print('影像日期', ee.Date(s2One.get('system:time_start')));

// 真彩色显示: 红、绿、蓝分别对应 B4、B3、B2。
Map.addLayer(
    s2One.divide(10000).clip(roi),
    {bands: ['B4', 'B3', 'B2'], min: 0, max: 0.3},
    '单景 Sentinel-2 真彩色'
);

// ----- 5. ImageCollection: 具有相同意义的一组时序影像 -----
var s2Collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate('2025-01-01', '2026-01-01')
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 30));

print('Sentinel-2 ImageCollection', s2Collection);
print('影像数量', s2Collection.size());
print('最小云量', s2Collection.aggregate_min('CLOUDY_PIXEL_PERCENTAGE'));

// median() 沿时间维逐像元取中位数, 得到一张合成 Image。
var medianImage = s2Collection.median().divide(10000).clip(roi);
Map.addLayer(
    medianImage,
    {bands: ['B8', 'B4', 'B3'], min: 0, max: 0.45},
    '年度中位数假彩色'
);

```

逐行阅读重点:

- `get()` 读取 Feature 或 Image 的单个属性。
- `aggregate_mean()` 对 FeatureCollection 属性列汇总。
- `first()` 从 ImageCollection 取单景, 仍可能是空值; 先用 `size()` 检查。
- `bandNames()` 查看影像波段。
- `median()` 沿时间维逐像元聚合, 把 ImageCollection 变为 Image。

2.2.2 结果说明

Console 显示五类对象结构、样地数量、平均郁闭度、单景波段和集合影像数；地图显示 ROI、样地点、单景真彩色和年度中位数假彩色。类型决定可调用的方法。遇到方法错误时，先打印对象，不要靠猜测修改。

2.2.3 常见报错解决

- 单景显示报 null：筛选条件无结果；打印 size()，放宽日期或云量。
- Sentinel-2 显示过亮：原始 SR 值需乘 0.0001 或除以 10000。
- aggregate_mean 返回空：属性不存在、拼写错误或值不是数值。

2.3 筛选、去云、镶嵌与裁剪

2.3.1 知识点讲解

影像筛选一般按范围、时间和元数据属性进行：

- filterBounds(roi)：保留与研究区相交的影像。
- filterDate(start, end)：开始包含、结束不包含。
- filter(ee.Filter.lt(property, value))：按整景云量等属性初筛。

整景云量筛选不能替代逐像元去云。课程用 Sentinel-2 SCL 去除云影、中高概率云、卷云和雪冰。mosaic() 是按集合顺序覆盖，适合同期拼接；median() 对时序取中位数，通常更能抑制残云；clip() 只保留研究区显示/输出范围。

本章与指数章节共用以下脚本，先重点阅读第 1—5 段。

```
// =====  
// GEE 从零入门课程：筛选、去云、镶嵌、裁剪与常用指数  
// 数据：Sentinel-2 SR Harmonized  
// =====  
  
// 1. 定义研究区与分析时段。  
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);  
var startDate = '2025-01-01';  
var endDate = '2026-01-01'; // filterDate 的结束日期不包含当天。  
  
// 2. SCL 去云函数。  
function maskS2(image) {  
  // SCL 是场景分类波段；下列类别需要剔除。  
  var scl = image.select('SCL');  
  var clearMask = scl.neq(0) // 0: 无数据  
    .and(scl.neq(1)) // 1: 饱和或坏像元  
    .and(scl.neq(3)) // 3: 云影  
    .and(scl.neq(8)) // 8: 中概率云  
    .and(scl.neq(9)) // 9: 高概率云  
    .and(scl.neq(10)) // 10: 卷云  
    .and(scl.neq(11)); // 11: 雪/冰  
  
  // 选出常用波段、应用掩膜、缩放为 0—1 反射率，并复制时间属性。  
  return image  
    .updateMask(clearMask)  
    .select(['B2', 'B3', 'B4', 'B8', 'B11'],
```

```

        ['blue', 'green', 'red', 'nir', 'swir1'])
        .multiply(0.0001)
        .copyProperties(image, ['system:time_start']);
    }

// 3. 依次执行范围、时间、整景云量筛选，再做逐像元去云。
var s2 = ee.ImageCollection("COPERNICUS/S2_SR_HARMONIZED")
    .filterBounds(roi)
    .filterDate(startDate, endDate)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 40))
    .map(maskS2);

print('去云后的影像集合', s2);
print('筛选后的影像数量', s2.size());

// 4. mosaic() 按集合顺序覆盖；适合同一天或人为排序后的拼接。
var mosaicImage = s2.sort('system:time_start').mosaic().clip(roi);
Map.addLayer(
    mosaicImage,
    {bands: ['red', 'green', 'blue'], min: 0, max: 0.3},
    'mosaic 镶嵌',
    false
);

// 5. median() 对时序逐像元取中位数，通常比直接 mosaic 更稳健。
var composite = s2.median().clip(roi);
Map.centerObject(roi, 9);
Map.addLayer(
    composite,
    {bands: ['red', 'green', 'blue'], min: 0.02, max: 0.30},
    '中位数真彩色'
);

// 6. NDVI = (NIR - Red) / (NIR + Red)。
var ndvi = composite.normalizedDifference(['nir', 'red']).rename('NDVI');

// 7. EVI = 2.5*(NIR-Red)/(NIR+6*Red-7.5*Blue+1)。
var evi = composite.expression(
    '2.5 * (NIR - RED) / (NIR + 6 * RED - 7.5 * BLUE + 1)',
    {
        NIR: composite.select('nir'),
        RED: composite.select('red'),
        BLUE: composite.select('blue')
    }
).rename('EVI');

// 8. NDBI = (SWIR1 - NIR) / (SWIR1 + NIR)。
var ndbi = composite.normalizedDifference(['swir1', 'nir']).rename('NDBI');

// 9. NDWI (McFeeters) = (Green - NIR) / (Green + NIR)。
var ndwi = composite.normalizedDifference(['green', 'nir']).rename('NDWI');

// 10. SAVI = 1.5*(NIR-Red)/(NIR+Red+0.5)，适合稀疏植被。
var savi = composite.expression(
    '1.5 * (NIR - RED) / (NIR + RED + 0.5)',
    {
        NIR: composite.select('nir'),
        RED: composite.select('red')
    }
).rename('SAVI');

// 11. 设置显示参数并逐层添加。
var vegetationPalette = [
    '8c510a', 'd8b365', 'f6e8c3', 'c7eae5', '5ab4ac', '01665e'
];
Map.addLayer(ndvi, {min: -0.2, max: 0.9, palette: vegetationPalette}, 'NDVI');
Map.addLayer(evi, {min: -0.1, max: 0.8, palette: vegetationPalette}, 'EVI', false);
Map.addLayer(ndbi, {min: -0.5, max: 0.5, palette: ['006837', 'FFFFBF', 'A50026']}, 'NDBI', false);

```

```

Map.addLayer(ndwi, {min: -0.5, max: 0.5, palette: ['A6611A', 'F5F5F5', '018571']}, 'NDWI', false);
Map.addLayer(savi, {min: -0.1, max: 0.8, palette: vegetationPalette}, 'SAVI', false);

// 12. 把五个指数合成多波段影像，便于统一统计或导出。
var indices = ndvi.addBands([evi, ndbi, ndwi, savi]);
print('五指数多波段影像', indices);

// 13. 在研究区内计算均值、最小值和最大值。
var reducer = ee.Reducer.mean()
  .combine({reducer2: ee.Reducer.min(), sharedInputs: true})
  .combine({reducer2: ee.Reducer.max(), sharedInputs: true});

var statistics = indices.reduceRegion({
  reducer: reducer,
  geometry: roi,
  scale: 20,
  maxPixels: 1e9,
  tileScale: 4
});
print('研究区指数统计', statistics);

```

结果说明：Console 返回去云后集合和影像数量；地图默认显示年度中位数真彩色。Layer Manager 中可打开 mosaic 比较，两者可能在残云、接边和日期代表性上不同。标准顺序是“范围/时间/元数据初筛 → 像元质量掩膜 → 波段选择和缩放 → 合成 → 裁剪”。尽早筛选能减少后续计算量。

2.3.2 常见报错解决

- **Pattern 'SCL' did not match any bands**：使用了不含 SCL 的 Sentinel-2 L1C 集合；改用 **COPERNICUS/S2_SR_HARMONIZED**。
- 去云后全空：阈值过严或研究区云多；逐项暂时关闭掩膜条件定位。
- mosaic 仍有云：mosaic 不自动识别质量；先去云或使用质量镶嵌。

2.4 NDVI、EVI、NDBI、NDWI 与 SAVI

2.4.1 指数原理

- $NDVI = (NIR - Red) / (NIR + Red)$ ：绿色植被活力的常用指标，受饱和、土壤背景和大气影响。
- $EVI = 2.5(NIR - Red) / (NIR + 6Red - 7.5Blue + 1)$ ：对高生物量区和大气背景有一定改进。
- $NDBI = (SWIR1 - NIR) / (SWIR1 + NIR)$ ：常用于建成区增强，但裸地也可能响应高。
- $NDWI = (Green - NIR) / (Green + NIR)$ ：本课程采用 McFeeters 水体指数；文献中“NDWI”也可能指 Gao 的植被含水指数，必须写清定义。
- $SAVI = 1.5(NIR - Red) / (NIR + Red + 0.5)$ ：通过土壤调节项减弱稀疏植被的土壤背景影响。

2.4.2 可运行代码

完整代码已在第 9 章脚本中给出。以下是最小指数模板：

```

// composite 的波段已统一命名并缩放为反射率。
var ndvi = composite.normalizedDifference(['nir', 'red']).rename('NDVI');
var evi = composite.expression(
  '2.5 * (NIR - RED) / (NIR + 6 * RED - 7.5 * BLUE + 1)',
  {NIR: composite.select('nir'), RED: composite.select('red'), BLUE: composite.select('blue')}
).rename('EVI');

```



```
var ndbi = composite.normalizedDifference(['swir1', 'nir']).rename('NDBI');
var ndwi = composite.normalizedDifference(['green', 'nir']).rename('NDWI');
var savi = composite.expression(
  '1.5 * (NIR - RED) / (NIR + RED + 0.5)',
  {NIR: composite.select('nir'), RED: composite.select('red')}
).rename('SAVI');
```

结果说明：森林在 NDVI/EVI/SAVI 中通常为较高正值；水体在本课程 NDWI 中通常较高；建成区在 NDBI 中可能较高。具体阈值随传感器、季节、区域和预处理改变，不能照搬。

指数是波段信息的压缩表达，不是地物类别本身。科研写作必须写明公式、波段、数据层级、合成方式和时间窗。

2.4.3 常见报错解决

- 指数范围远超 -1—1：反射率缩放或公式括号错误。
- 波段反了导致森林为负：检查 `normalizedDifference([nir, red])` 的顺序。
- NDWI 结果和论文相反：论文可能使用 NIR/SWIR 定义；先核对公式而不是只看名称。

2.5 均值、最大值、最小值与月度/年度合成

2.5.1 知识点讲解

`ImageCollection.mean()`、`median()`、`min()`、`max()` 沿时间维逐像元聚合。`reduceRegion()` 在空间范围内把一张影像压缩成字典。两者都叫“统计”，但方向不同：前者跨时间，后者跨空间。

月度或年度合成通常用 `ee.List.sequence()` 创建月份/年份，再通过 `map()` 为每个时间段筛选和合成。合成影像应设置 `system:time_start`，否则时序图没有正确横轴。

2.5.2 完整可运行代码

```
// =====
// GEE 从零入门课程：月度/年度合成、图表与三类导出
// =====

var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
Map.centerObject(roi, 9);

// 1. 去云并添加 NDVI。
function prepareS2(image) {
  var scl = image.select('SCL');
  var mask = scl.neq(0)
    .and(scl.neq(1))
    .and(scl.neq(3))
    .and(scl.neq(8))
    .and(scl.neq(9))
    .and(scl.neq(10))
    .and(scl.neq(11));

  var reflectance = image.updateMask(mask)
    .select(['B2', 'B3', 'B4', 'B8', 'B11'],
      ['blue', 'green', 'red', 'nir', 'swir1'])
    .multiply(0.0001);

  var ndvi = reflectance
    .normalizedDifference(['nir', 'red'])
    .rename('NDVI');
```

```

return reflectance.addBands(ndvi)
    .copyProperties(image, ['system:time_start']);
}

var collection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(roi)
    .filterDate('2024-01-01', '2026-01-01')
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 40))
    .map(prepareS2);

// 2. 生成 2025 年 12 个月的 NDVI 中位数合成。
var months = ee.List.sequence(1, 12);
var monthlyNdvi = ee.ImageCollection.fromImages(
    months.map(function(month) {
        month = ee.Number(month);
        var start = ee.Date.fromYMD(2025, month, 1);
        var end = start.advance(1, 'month');
        return collection
            .filterDate(start, end)
            .select('NDVI')
            .median()
            .rename('NDVI')
            .set('month', month)
            .set('system:time_start', start.millis());
    })
);
print('2025 月度 NDVI 合成', monthlyNdvi);

// 3. 生成 2024—2025 年年度最大 NDVI 合成。
var years = ee.List.sequence(2024, 2025);
var annualMaxNdvi = ee.ImageCollection.fromImages(
    years.map(function(year) {
        year = ee.Number(year);
        var start = ee.Date.fromYMD(year, 1, 1);
        var end = start.advance(1, 'year');
        return collection
            .filterDate(start, end)
            .select('NDVI')
            .max()
            .rename('NDVI')
            .set('year', year)
            .set('system:time_start', start.millis());
    })
);
print('年度最大 NDVI 合成', annualMaxNdvi);

// 4. 显示 2025 年最大 NDVI。
var ndvi2025 = annualMaxNdvi
    .filter(ee.Filter.eq('year', 2025))
    .first()
    .clip(roi);
Map.addLayer(
    ndvi2025,
    {min: 0.2, max: 0.9, palette: ['F7FCB9', '78C679', '006837']},
    '2025 最大 NDVI'
);

// 5. 研究区月度 NDVI 折线图。
var monthlyChart = ui.Chart.image.series({
    imageCollection: monthlyNdvi,
    region: roi,
    reducer: ee.Reducer.mean(),
    scale: 100,
    xProperty: 'system:time_start'
}).setOptions({
    title: '2025 年研究区月度平均 NDVI',
    hAxis: {title: '月份', format: 'MM'},
    vAxis: {title: 'NDVI', viewWindow: {min: 0, max: 1}},

```

```

    lineWidth: 2,
    pointSize: 5,
    colors: ['2E7D32']
  });
  print(monthlyChart);

  // 6. NDVI 直方图。
  var histogram = ui.Chart.image.histogram({
    image: ndvi2025,
    region: roi,
    scale: 100,
    maxBuckets: 40
  }).setOptions({
    title: '2025 年最大 NDVI 频率分布',
    hAxis: {title: 'NDVI'},
    vAxis: {title: '像元频数'},
    colors: ['388E3C']
  });
  print(histogram);

  // 7. 导出影像到 Google Drive。运行脚本后到 Tasks 标签点击 Run。
  Export.image.toDrive({
    image: ndvi2025.toFloat(),
    description: 'NDVI_2025_Max',
    folder: 'GEE_Course',
    fileNamePrefix: 'ndvi_2025_max',
    region: roi,
    scale: 20,
    maxPixels: 1e13,
    fileFormat: 'GeoTIFF',
    formatOptions: {cloudOptimized: true}
  });

  // 8. 导出矢量研究区为 SHP。
  var roiFeature = ee.Feature(roi, {name: 'Xishuangbanna_demo_roi'});
  Export.table.toDrive({
    collection: ee.FeatureCollection([roiFeature]),
    description: 'ROI_Vector',
    folder: 'GEE_Course',
    fileNamePrefix: 'roi_vector',
    fileFormat: 'SHP'
  });

  // 9. 把年度区域均值转换为表格，再导出 CSV。
  var annualTable = annualMaxNdvi.map(function(image) {
    var meanNdvi = image.reduceRegion({
      reducer: ee.Reducer.mean(),
      geometry: roi,
      scale: 100,
      maxPixels: 1e9,
      tileScale: 4
    }).get('NDVI');

    return ee.Feature(null, {
      year: image.get('year'),
      mean_ndvi: meanNdvi
    });
  });
  print('年度 NDVI 表格', annualTable);

  Export.table.toDrive({
    collection: annualTable,
    description: 'Annual_NDVI_Table',
    folder: 'GEE_Course',
    fileNamePrefix: 'annual_ndvi_table',
    fileFormat: 'CSV',
    selectors: ['year', 'mean_ndvi']
  });

```

2.5.3 逐行阅读重点

1. `ee.ImageCollection.fromImages()` 把服务器端影像列表变成集合。
2. `start.advance(1, 'month')` 安全处理不同月份天数。
3. 年度最大 NDVI 是每个像元在该年可用观测中的最大值，不一定来自同一日期。
4. 区域统计必须明确 `scale`；合成影像若不指定尺度，默认投影可能带来意外结果。

结果说明：Console 出现 12 景月度合成、2 景年度合成和年度统计表；地图显示 2025 最大 NDVI；Console 还显示月度折线图和直方图；Tasks 创建三类导出任务。

时间合成解决“每个时间段一张代表影像”，空间统计解决“每个区域一个或多个数值”。科研分析通常需要两者结合。

2.5.4 常见报错解决

- 某月影像为空：云多或筛选太严；先统计每月 `size()`，必要时延长窗口。
- 年度最大值异常偏高：可能残云、雪或边缘噪声；改进质量掩膜并检查 Inspector。
- `Too many pixels`：提高 `scale`、增加 `maxPixels` 或缩小区域；不要无原则把 `maxPixels` 调到极大。

2.6 图层配色、可视化与 Inspector 判读

2.6.1 知识点讲解

可视化参数不会修改分析值。常用参数包括 `bands`、`min`、`max`、`palette`、`gamma` 和 `opacity`。RGB 影像用三个波段；单波段指数用连续调色板。变化图使用以零为中心的发散色带，类别图使用离散色。

推荐配色原则：

1. NDVI：低值棕黄、高值绿色。
2. 趋势：退化红、零值浅色、改善绿。
3. 水体：深蓝或青色，但不要把“蓝色=水”当分析结论。
4. 色盲友好：避免只靠红绿区分，增加图例文字和类别编码。

2.6.2 可运行代码

```
var ndvi = ee.ImageCollection('MODIS/061/MOD13A2')
  .filterDate('2025-01-01', '2025-01-17')
  .first()
  .select('NDVI')
  .multiply(0.0001);
var vis = {
  min: -0.2,
  max: 0.9,
  palette: ['8C510A', 'D8B365', 'F6E8C3', '5AB4AC', '01665E']
};
Map.setCenter(100.8, 22.0, 7);
Map.addLayer(ndvi, vis, 'MODIS NDVI');
```

```
print('打开 Inspector 后点击地图查看原始 NDVI 值');
```

结果说明：地图显示 NDVI 连续色带。打开 Inspector，点击不同地物，比较像元原值与颜色；修改 min/max 后颜色会变化，但 Inspector 数值不变。

2.6.3 知识点总结

分析值、显示拉伸和最终制图是三个层次。论文必须说明分析阈值，不能从屏幕颜色反推数值。

2.6.4 常见报错解决

- 调色板顺序反了：交换 palette 顺序，不要把结果乘以 -1 来“修图”。
- 图层颜色一片：min/max 范围过宽或过窄；先看直方图。
- Inspector 显示 masked：该像元被去云/森林掩膜剔除，或当前图层分辨率不同。

2.7 ui.Chart 图表绘制

2.7.1 知识点讲解

ui.Chart.image.series() 对单个区域绘制 ImageCollection 时间序列；seriesByRegion() 比较多个分区；ui.Chart.image.histogram() 查看单张影像的值分布；ui.Chart.feature.groups() 对长表按分类字段分组作图。图表通过 print(chart) 显示在 Console。

2.7.2 可运行代码

```
var roi = ee.Geometry.Rectangle([100.55, 21.72, 101.10, 22.22]);
var modis = ee.ImageCollection('MODIS/061/MOD13A2')
  .filterBounds(roi)
  .filterDate('2024-01-01', '2026-01-01')
  .select('NDVI')
  .map(function(image) {
    return image.multiply(0.0001)
      .rename('NDVI')
      .copyProperties(image, ['system:time_start']);
  });

var chart = ui.Chart.image.series({
  imageCollection: modis,
  region: roi,
  reducer: ee.Reducer.mean(),
  scale: 1000,
  xProperty: 'system:time_start'
}).setOptions({
  title: '研究区 MODIS NDVI 时间序列',
  hAxis: {title: '日期'},
  vAxis: {title: 'NDVI'},
  lineWidth: 1,
  pointSize: 2,
  colors: ['#2E7D32']
});
print(chart);
```

结果说明：Console 显示两年 NDVI 折线。季节性植被会形成周期波动；异常低值可能是物候、干旱、扰动或残余质量问题，需结合影像和质量波段解释。图表是质量控制工具，也是科研结果表达工具。先用图表检查时序，再做趋势分析。

2.7.3 常见报错解决

- 图表没有日期：映射函数丢失 `system:time_start`；使用 `copyProperties`。
- 图表过慢：区域过大且 `scale` 太细；图表只用于概览时提高 `scale`。
- 折线断裂：某些时点区域全部被掩膜；检查云掩膜与数据覆盖。

2.8 导出影像、矢量和表格

2.8.1 知识点讲解

`Export.image.toDrive()` 导出栅格，`Export.table.toDrive()` 同时用于矢量和属性表。代码运行只创建未提交任务；必须在 Tasks 点击 Run。任务提交后关闭页面不会丢失，未提交任务则会随页面关闭丢失。

影像导出关键参数：

- `image`：待导出影像。
- `region`：导出范围。
- `scale`：名义像元大小，不能把 30 m Landsat 设为 10 m 就当成真实 10 m 信息。
- `crs/crsTransform`：论文需要严格网格对齐时显式指定。
- `maxPixels`：允许的最大像元数。
- `formatOptions.cloudOptimized`：生成 Cloud Optimized GeoTIFF。

表格导出关键参数：`collection`、`fileFormat`、`selectors`。Shapefile 字段名和字符编码限制较多；复杂属性优先 GeoJSON/CSV，空间分析优先 GeoPackage（需在本地转换）或 Earth Engine Asset。

2.8.2 可运行代码

完整示例见第 11 章脚本的第 7—9 段。最小模板如下：

```
Export.image.toDrive({
  image: ndvi2025.toFloat(),
  description: 'NDVI_2025_Max',
  folder: 'GEE_Course',
  region: roi,
  scale: 20,
  maxPixels: 1e13,
  fileFormat: 'GeoTIFF',
  formatOptions: {cloudOptimized: true}
});

Export.table.toDrive({
  collection: annualTable,
  description: 'Annual_NDVI_Table',
  folder: 'GEE_Course',
```

```
fileFormat: 'CSV',  
selectors: ['year', 'mean_ndvi']  
});
```

结果说明：Tasks 中出现任务。点击 Run，核对文件名、Drive 文件夹、范围、分辨率和 CRS，再提交。完成后到 Google Drive 指定文件夹查看。

显示正确不等于导出正确。提交前检查单位、数据类型、掩膜、范围、分辨率、CRS、NoData 和波段名。

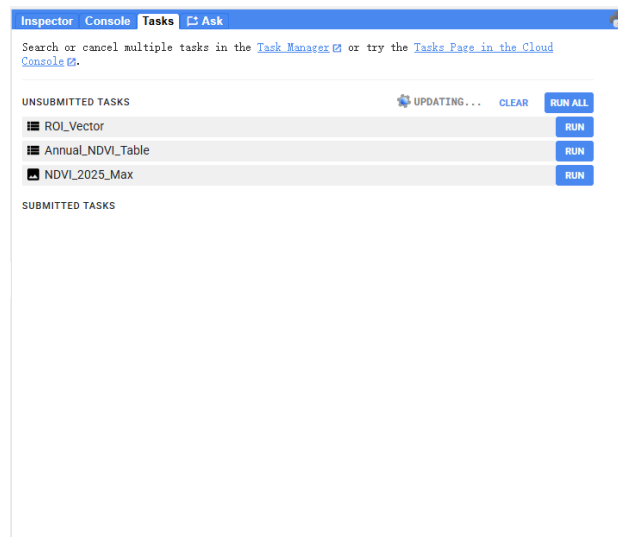
2.8.3 导出技巧 - Google Earth Engine 一键 RUN

在批量下载数据的时候，会有很多 run 的按钮，需要一个一个去点，很麻烦，类似于下图：



使用国外作者制作的一个谷歌浏览器的插件：Open Earth Engine extension，可以一键导出所有需要下载的文件。[下载链接](#)

当安装好该插件后刷新 GEE 页面，即可在 Task 界面看到 RUN ALL 按钮，非常方便有效。



2.8.4 常见报错解决

- **Export too large**: 缩小区域、提高 scale、分区导出或减少波段。
- 输出影像偏移: 为严格对齐使用参考影像的 `crs` 和 `crsTransform`, 不要只依赖 `scale`。
- CSV 少了几何: CSV 默认主要是属性; 需要空间可导出 GeoJSON/SHP, 或在 `selectors` 加 `.geo` (格式支持时)。

2.9 客户端/服务器、map 与性能调试

2.9.1 知识点讲解

普通 JavaScript 值 (字符串、数组、数字) 在客户端; `ee.*` 对象在服务器。服务器端 `map(function(element){...})` 中不能使用客户端 `if` 读取服务器属性, 也不要再在映射函数中调用 `print()` 或 `getInfo()`。

性能优化顺序:

1. 尽早 `filterBounds/filterDate/select`。
2. 不重复计算同一复杂表达式; 保存中间对象描述。
3. 交互统计提高 `scale`; 最终导出再用目标尺度。
4. `reduceRegion/reduceRegions` 明确 `scale`、`maxPixels`, 内存不足时尝试 `tileScale: 2` 或 `4`。
5. 使用 Profiler 找到最耗时的算法, 而不是凭感觉重写全部代码。

2.9.2 错误与正确示例

```
var years = ee.List.sequence(2020, 2025);

// 正确: 在服务器端 map 中把元素显式转换为 ee.Number。
var labels = years.map(function(year) {
  year = ee.Number(year);
  return ee.String('year_').cat(year.format('%d'));
});
print('服务器端年份标签', labels);

// 客户端固定参数可使用普通 if; 完整项目用它切换 S2/Landsat。
var source = 'S2';
if (source === 'S2') {
  print('使用 Sentinel-2');
} else {
  print('使用 Landsat');
}
```

结果说明: Console 返回服务器端列表和数据源文字。尝试把 `year` 直接与客户端字符串做不支持的运算, 会出现类型或序列化错误。

GEE 编程的难点不是 JavaScript 语法本身, 而是理解“计算在哪里执行”。链式服务器对象是主角, 客户端只负责组织参数和界面。

2.9.3 常见报错解决

- **A mapped function's arguments cannot be used in client-side operations:** 把客户端运算改为 `ee.Number/ee.String/ee.Algorithms.If` 等服务器表达式。
- **User memory limit exceeded:** 提高 `tileScale`、减少波段/区域/时段，或改用批导出。
- **Too many concurrent aggregations:** 减少在 `map` 内同时执行的大量 `reduce`，分批计算。

2.10 综合练习

2.10.1 任务

以自己的研究区完成以下操作：

1. 绘制或定义一个 ROI。
2. 获取 2025 年 Sentinel-2 SR Harmonized，整景云量小于 30%。
3. SCL 去云，计算 NDVI 和 NDWI。
4. 生成中位数合成和最大 NDVI。
5. 绘制月度 NDVI 区域均值图。
6. 导出最大 NDVI GeoTIFF 和月度均值 CSV。

2.10.2 验收标准

- Console 中影像数大于 0。
- 波段名称和缩放正确，NDVI 大体在 -1—1。
- 时间图横轴是日期且没有错误的 1970 年。
- Tasks 中影像和表格任务参数完整。
- 代码包含数据集 ID、ROI、时间、云量阈值、scale 和导出范围。

2.10.3 常见报错解决

- 替换 ROI 后超时：先用较小 ROI 和 100 m 统计尺度验证逻辑。
- 自己的矢量资产无权限：Assets 中复制 ID，并确认当前账号和 Cloud 项目。
- 月度表有 null：该月没有有效观测；在输出中保留缺测并说明，不建议随意用 0 填补。